Overview

This project implements a simplified Git-like version control system in C. It demonstrates key filesystem and versioning concepts such as object storage, staging, tree structures, and commit history.

Screenshots

Phase 1

1A – Object Tests

```
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B – Object Storage Layout

```
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2

2A – Tree Tests

```
c object.c index.c -o test_tree -lcrypto
./test_tree
object.c: In function 'object_write':
object.c:161:52: warning: '%s' directive output may be truncated writing up to 62 bytes into a region of size between
4 and 515 [-Wformat-truncation=]
  161 |     snprintf(temp_path, sizeof(temp_path), "%s/tmp_%s", shard_dir, hex + 2);
      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
In function 'snprintf',
    inlined from 'object_write' at object.c:161:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 6 and 579 bytes int
o a destination of size 520
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
index.c: In function 'index_load':
index.c:207:9: warning: '__builtin_strncpy' output may be truncated copying 511 bytes from a string of length 511 [-Ws
tringop-truncation]
  207 |         strncpy(entry->path, path, sizeof(entry->path) - 1);
      |         ^
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
```

2B – Raw Tree Object

```
xxd <paste-object-path> | head -n 20
-bash: syntax error near unexpected token `|'
```

Phase 3

3A – Init → Add → Status

```
echo "hello" > hello.txt
./pes add hello.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

3B – Index File

```
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776540433 6 hello.txt
```

Phase 4

4A – Commit Log

```
./pes commit -m "second commit"
./pes commit -m "third commit"
./pes log
Committed: 8996fb1f88aa... first commit
Committed: 30f6cfc0e70f... second commit
Committed: 06fc9fc84570... third commit
commit 06fc9fc8457057791fc21e80c76b22c00e10e0a28b59e18cd83b9d9113e85c0c
Author: PES User <pes@localhost>
Date:   1776540473

    third commit

commit 30f6cfc0e70f37cc70970b0a6d04420a5446b25bf0f5fff37e73ac0091de237a
Author: PES User <pes@localhost>
Date:   1776540473

    second commit

commit 8996fb1f88aa01989a1aa405e6ca5f3ceea6db8a1b04eb109a6bdf83a8352df9
Author: PES User <pes@localhost>
Date:   1776540473

    first commitirst commit<Q
```

4B – Object Growth

```
.pes/HEAD
.pes/index
.pes/objects/06/fc9fc8457057791fc21e80c76b22c00e10e0a28b59e18cd83b9d9113e85c0c
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/30/f6cfc0e70f37cc70970b0a6d04420a5446b25bf0f5fff37e73ac0091de237a
.pes/objects/89/96fb1f88aa01989a1aa405e6ca5f3ceea6db8a1b04eb109a6bdf83a8352df9
.pes/objects/eb/c17d3259211bc466654a87188efa62cb38d9d9a3aa32a9ebff60ee03958f8b
.pes/refs/heads/main
```

4C – HEAD + Branch

```
cat .pes/HEAD
06fc9fc8457057791fc21e80c76b22c00e10e0a28b59e18cd83b9d9113e85c0c
ref: refs/heads/main
```

Final Integration

```
--- Running integration tests ---
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: af4ff046973e... Initial commit

Log after first commit:
commit af4ff046973ebe58b264396c6aa880be8953718471f228d2f316253ff76de3a6
Author: PES User <pes@localhost>
Date:   1776540746

    Initial commit
```

```
--- Second Commit ---
Committed: 26efea989039... Update file.txt

--- Third Commit ---
Committed: 13401bd89ca0... Add farewell

--- Full History ---
commit 13401bd89ca054767e49949e059fb38f94117e51abf0ae6ae34618b5486182cb
Author: PES User <pes@localhost>
Date:   1776540747

    Add farewell

commit 26efea98903975275f24a3d151598a0aaf4d1283861b0d1a0ea9c4867aa23937
Author: PES User <pes@localhost>
Date:   1776540746

    Update file.txt

commit af4ff046973ebe58b264396c6aa880be8953718471f228d2f316253ff76de3a6
Author: PES User <pes@localhost>
Date:   1776540746

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
13401bd89ca054767e49949e059fb38f94117e51abf0ae6ae34618b5486182cb

--- Object Store ---
Objects created:
10
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/13/401bd89ca054767e49949e059fb38f94117e51abf0ae6ae34618b5486182cb
.pes/objects/26/efea98903975275f24a3d151598a0aaf4d1283861b0d1a0ea9c4867aa23937
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/af/4ff046973ebe58b264396c6aa880be8953718471f228d2f316253ff76de3a6
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Analysis Questions

Q5.1 – Branching & Checkout

To implement pes checkout <branch>:

Files that change:

- .pes/HEAD → updated to point to new branch
- .pes/refs/heads/<branch> → read commit hash
- Working directory → updated to match tree of target commit

Steps:

1. Read commit hash from target branch
2. Load its tree
3. Replace working directory files with tree contents
4. Update HEAD

Complexity:

- Need to safely overwrite files
- Must preserve untracked files
- Must avoid data loss
- Requires reconstructing directory structure from tree objects

---

Q5.2 – Dirty Working Directory Detection

To detect conflicts:

1. Compare:
    - current working file
    - index entry
    - target branch version
2. If:
    - file is modified (working ≠ index)
    - AND target version differs

Then checkout must fail

Implementation:

- Use stat() to check file metadata
- Compare stored hash in index vs recomputed hash

Q5.3 – Detached HEAD

Detached HEAD means:

- HEAD points directly to a commit (not a branch)

What happens:

- New commits are created
- But no branch references them

These commits can become unreachable

Recovery:

- Use commit hash (from log)
- Create new branch:
- pes branch new-branch <commit-hash>

Q6.1 – Garbage Collection

Goal:

Remove unreachable objects

Algorithm:

1. Start from:
    - all branch heads
2. Traverse:
    - commits → trees → blobs
3. Mark all reachable objects
4. Delete:
    - objects NOT in reachable set

Data structure:

- **Hash set (unordered set)** for visited hashes

Estimate:

- 100,000 commits
- Each commit → 1 tree + multiple blobs

Q6.2 – GC Race Condition

Problem:

GC deletes objects while commit is being created

Example:

1. Commit writes object
2. GC runs before commit is linked to HEAD
3. GC deletes object → data loss

Git's Solution:

- Uses **reference counting**
- Uses **temporary files**
- Uses **lock files**
- Uses **grace period before deletion**

Ensures objects are not deleted prematurely